Prepared naming, Access and Notice candidate — 2 November 2026. Build, signing, installed acceptance and publication remain pending. Earlier signed releases and receipts apply only to their identified artifacts.

Townlight Records

Townlight Beta 1 · Naming candidate; component versions advanced · AGPL-3.0-only

Records manages request intake, responsive evidence, review and the final response. Material received from another Townlight product is evidence for a request. Records separately controls release decisions, response approval, exports and fulfillment.

Start here

Staff should use the operating manual. Administrators should read first-run, configuration and recovery before changing a deployment. Developers should read the exact API/schema reference and matching source companion. This edition describes the prepared source candidate; obtain its installer only after signing and installed acceptance identify the exact artifact.

- Operating manual · Printable PDF · Editable Word manual
- API and data dictionary · OpenAPI JSON
- Product and suite directory · Terms and full license

Exact version and candidate

Identity	Value
Documentation	Townlight Beta 1 naming, Access and Notice candidate
Intended installer version	1.7.0.1; build and acceptance pending
Native desktop source version	1.7.0-1
Product component	townlight-records 1.8.0.dev2
Principal import package	townlight_records
Candidate hashes	Pending build, signing and installed acceptance
License	AGPL-3.0-only

Use a fresh profile and empty database. Existing unmarked city data is refused before initialization; no in-place upgrade is supported. Preserve the existing `%LOCALAPPDATA%\CivicSuite` profile, data and recovery material. Historical releases retain their original identities.

What staff can do

- **Receive and identify the request:** Create the request with its requester, description and recorded deadline basis. Save it and retain the request identifier. Record assignment, clarification and work in the request history.

- **Find responsive evidence:** Search only the permitted sources. Inspect the source title, date, citation, visibility and version before attaching responsive material. Open the saved attachment to confirm that it is the intended evidence.
- **Preserve the original:** Keep original bytes and their hash. Add a separately reviewed release-ready or redacted copy when needed. Renaming a private file does not remove private content.
- **Record each release decision:** Explicitly record release, redact or exempt for the relevant document, with the applicable reason and source. Do not treat an AI suggestion or another product's review as the Records legal decision.
- **Draft and approve:** Draft the response and review the selected attachments. Record approval of the exact response text. Editing that text changes its hash and requires a new approval.
- **Export both artifacts:** Export the approved response and the release package. If release is refused, inspect the missing decision, approval or source-permission message. Resolve that condition before trying again.
- **Fulfill and retain:** Use the explicit fulfillment action only after checking the outgoing artifacts. Retain the response, package and history. A preview, attachment or export by itself does not fulfill the request.

Operating procedures and examples

The operating manual now includes these existing procedures, with synthetic inputs, readback and limits. Follow the actual app boundary identified there; a standalone source capability is not automatically a desktop screen.

- Fees messages notifications and local intake.
- Record a fee line or waiver.
- Save staff and requester messages.
- Configure templates and inspect the outbox.
- Exercise existing local public intake and tracking.

Architecture and data

Runtime and storage

This prepared source candidate targets **Windows x64 fresh installation**. Its native shell uses **Rust 2021, Tauri 2 and Microsoft WebView2**; browser assets are JavaScript built with **Vite 8.1.6**. This describes the native desktop frontend; Station has a separate React operator, specified below or in its product manual. Canonical business workflows use bundled **CPython 3.13.5, FastAPI, Pydantic, SQLAlchemy and Alembic**, with **PostgreSQL 17 / PostGIS** and managed original files. The prior accepted runtime contained FastAPI 0.162.0; this changed candidate requires its own installed dependency verification. Package minimum Python requirements can differ from the bundled executable; the metadata reference preserves both. No stack rewrite is implied by the beta label.

The native transport is fixed to loopback `127.0.0.1:15480`; PostgreSQL uses the configured local runtime port (default `15432`). Optional local model service defaults to `127.0.0.1:15434`. Those are deployment defaults, not internet endpoints. The runtime has health, module and readiness endpoints; readiness is stronger than process existence. The task queue is the existing local Core task infrastructure, not an assertion that a separate Redis service is required for this desktop composition.

The default native profile is `%LOCALAPPDATA%\CivicSuite`; the historical directory name remains part of compatibility. Do not rename it to Townlight. A configured `TOWNLIGHT_PROFILE_ROOT` must be absolute. Local configuration and secrets live under that profile; protected managed files and database state must be backed up coherently. Source/runtime identity is distinct from a product's display label.

Data and consistency

Requests; assignments and deadlines; search sessions; evidence attachments and original hashes; release copies; exemption decisions; messages; fee line items; response approvals; export artifacts and audit events.

Versioned saves use expected prior revision identifiers where defined. Reopen and reconcile on conflict. Exact exports and handoffs use source IDs, current visibility, revision and SHA256. Review belongs to the saved revision and does not transfer silently to edited text. Database changes use the owning product's migrations; a request or readiness check is not an authorization to mutate schema.

Capacity and platform statements

This beta has synthetic single-profile workflow evidence. It does not have a published benchmark for concurrent staff capacity, minimum RAM/CPU, large-source limits or a field SLA. Do not turn a developer machine's specification into a supported minimum. The new installer size remains pending its build; allow additional disk for extracted runtime, database, originals, exports, media and backups. Select equipment/storage after sizing those actual workloads.

The exact runtime payload lock records pinned files, sizes and hashes. The component metadata records shipped dependency declarations. These references are more precise than a manually copied dependency list. Third-party and historical license notices must accompany their matching payloads.

Data contract	Description
Input	A request description and requester, authorized source documents, explicit cross-product evidence selections and a reviewer's decisions.
Retained records	Requests; assignments and deadlines; search sessions; evidence attachments and original hashes; release copies; exemption decisions; messages; fee line items; response approvals;

Data contract	Description
	export artifacts and audit events.
Output	A retained request history, reviewed attachments, approved response text and a release package after the applicable release checks.
Connections	Receives selected evidence from Meetings, Code, Zoning, Planning, Permitting, Inspections and Station. Station originals are locked private evidence. A product’s public copy remains subject to the Records response workflow.

Setup and operation

1. Obtain the identified signed private candidate and its corresponding-source companion from the beta coordinator. The landing pages in this documentation are not an installer download service.
2. On the intended fresh Windows test installation, verify the MSI hash and publisher signature before installing. Preserve unrelated profiles and recovery material. Installed acceptance for this source candidate remains pending; no in-place legacy upgrade is supported.
3. Launch Townlight under the intended named Windows account. Choose the storage folders. Select **Townlight Inspections candidate** (`inspections-preview`) for the bounded desktop composition, including Access and Notice. A narrower default selection will not expose all workspaces.
4. Enter the city profile, time zone and contacts. Create the first named local administrator, sign in and choose protected backup storage. **Continue without local AI** is valid for the deterministic workflows.
5. Run Health Check and wait for readiness. Open Townlight. In System Health select **Finish Records Migration** to activate the canonical Python/PostgreSQL runtime. Do not fabricate migration receipts or seed accounts.
6. Create named staff through Settings. Import authorized sources using each product’s source library. All examples in this package are synthetic; they are not municipal authority.
7. Provision Station separately if it is part of the beta exercise. Selecting the desktop composition does not start or configure Station. Configure vision explicitly only if automatic photo drafting is selected.

Configuration

Native deployment configuration

Use Setup, Settings and System Health for supported staff operation. Administrators provision folders, backup storage, named accounts and the city profile. Select `inspections-preview` for the bounded desktop composition, including Access and Notice. Station remains a separate

service boundary. Do not use a standalone product's demo server as a substitute for the installed workspace.

Technical deployments use `TOWNLIGHT_PRODUCT_PROFILE`, `TOWNLIGHT_PROFILE_ROOT`, `DATABASE_URL`, `TOWNLIGHT_RUNTIME_PORT`, `PORTAL_MODE` and the runtime-managed secret files. Treat database URLs and native session proofs as secrets. The installed native domain runtime explicitly refuses non-loopback binding. `PORTAL_MODE=private` is the accepted posture; changing it does not by itself commission an internet portal, authentication provider or reverse proxy.

The isolated documentation exporter sets a fictional profile and unusable synthetic database origin only to read schemas; those values are not deployment configuration. Keep deployment values in the intended process/account scope. An already-running process does not automatically inherit later environment edits.

Communications and review configuration

SMTP settings include host, port, username/password, sender address and TLS choice. Configure and verify actual delivery only for a deployment that selects external mail. Connector host allowlists and encrypted data-source configuration are separate controls. Records fee schedules concern the records workflow; they do not authorize Permitting to calculate an official permit fee. Keep resident portal posture and self-registration separate from named local staff setup.

API usage

Native command example

The following JavaScript belongs in the existing trusted Townlight frontend/development environment with `invoke` imported from `@tauri-apps/api/core`. It is not a command for an arbitrary web page or an instruction for nontechnical staff. IDs must come from records created in an isolated synthetic exercise.

```
import { invoke } from '@tauri-apps/api/core';
const result = await invoke('records_domain_request', {request: {operation:
'view', request_id: requestId}});
console.log(result);
```

Trusted local HTTP example

An administrator/developer can inspect an already-configured isolated runtime from a trusted local process. This PowerShell example reads the signed-in synthetic profile's existing proof into memory; it does not create a user, seed a session, reveal the proof in output or bypass a permission check. Do not use real city data in development examples. A read response does not prove approval or publication.

```
$profilePath = Join-Path $env:LOCALAPPDATA 'CivicSuite'
$sessionPath = Join-Path $profilePath 'config\local-session.json'
$session = Get-Content -LiteralPath $sessionPath -Raw | ConvertFrom-Json
$headers = @{ 'X-Townlight-Session' = $session.session_hash }
```

```
Invoke-RestMethod -Uri 'http://127.0.0.1:15480/desktop/v1/records-state'
-Headers $headers
Remove-Variable session, headers
```

A synthetic write with subsequent readback

This example demonstrates the existing Meetings HTTP contract in an isolated exercise. It is intentionally fictional. It does not start a service. Its authentication setup is included so it can be run independently under the signed-in synthetic account. If a write times out, inspect current records before repeating it.

```
$profilePath = Join-Path $env:LOCALAPPDATA 'CivicSuite'
$sessionPath = Join-Path $profilePath 'config/local-session.json'
$session = Get-Content -LiteralPath $sessionPath -Raw | ConvertFrom-Json
$headers = @{ 'X-Townlight-Session' = $session.session_hash }
$body = @{
  title = 'Synthetic documentation exercise'
  scheduled_start = '2026-11-05T18:00:00-07:00'
  location = 'Synthetic room'
  meeting_type = 'regular'
} | ConvertTo-Json
$meeting = Invoke-RestMethod -Method Post `
  -Uri 'http://127.0.0.1:15480/desktop/v1/meetings/meetings' `
  -Headers $headers -ContentType 'application/json' -Body $body
Invoke-RestMethod `
  -Uri ('http://127.0.0.1:15480/desktop/v1/meetings/meetings/' + $meeting.id)
`
  -Headers $headers
Remove-Variable session, headers
```

This write requires Meetings to be enabled and the signed-in staff role to permit it. It is a cross-suite integration example; for this product's request models, use the complete technical reference below.

MCP status

No direct product MCP server is exposed by the accepted Townlight desktop or Station boundary. Do not configure an invented `townlight mcp` command or assume an AI client has staff permissions. The integration surfaces for this beta are native IPC, the documented HTTP APIs, source bundle imports, exact exports and the existing Station CLI.

The bundled transcription code includes a **TextFlowKit MCP adapter** as a separate source capability. It offers job-based `transcribe_media`, `get_transcript`, `list_jobs`, `get_job_status` and `export_transcript`, and additional tools declared in that adapter. It uses stdio first and optional Streamable HTTP, with the MCP SDK as an optional extra. It is not automatically launched, authenticated as Townlight staff or accepted as a Townlight product MCP deployment. This documentation does not add such a service.

For a transcription client already provisioned separately, an illustrative tool call is:

```
{"name":"get_job_status","arguments":{"job_id":"synthetic-existing-job-id"}}
```

Use a job ID returned by that transcription service. This example does not create a Townlight record or bypass its review/Records handoff. The exact optional adapter tool signatures are supplied in the MCP source reference. Station meeting capture does not become live speech recognition because this optional donor adapter exists.

Verification and known limits

Product	Recorded result	Precisely exercised
Records	PASS	Intake, note, original attachment/readback, separate release copy, explicit release decision, draft/approval, approved response and release package exports, fulfillment/readback and native workspace.

These results come from the identified signed candidate's fresh normal-profile Sandbox exercise: real native IPC/HTTP/PostgreSQL and selected headless controls. All accounts, source bundles and media were synthetic. The guest install/uninstall and recovery passed, and the owned Sandbox stopped. No host installed application or standalone CivicCast was changed. This documentation task reuses those receipts; it does not claim a new software acceptance run.

The workflow matrix identifies raw receipt filenames. The evidence index retains their original hashes; raw receipts remain in the private beta handoff rather than being silently republished with the documentation site.

The limits remain: actual city field acceptance and equipment/network commissioning; Station automatic provisioning/SCM commissioning; every UI/role combination; field recognition accuracy; and historical in-place upgrades are not certified. They are not five invented feature requirements. Fresh-install controlled beta handoff remains the accepted scope. This package makes no city-deployment, publication or general-availability claim.

The selected staff workflow is accepted. A resident internet deployment, every jurisdiction's requirements, complete federation and exhaustive redaction/accessibility behavior are not certified by that acceptance. SMTP code exists; logging a message or outbox entry does not establish email delivery.

Development and repository map

Use the corresponding source for the candidate rather than assuming the public default branch matches it. The source companion contains `installed-owned-source`, product build inputs, desktop Rust/browser sources, locks and `BUILD.md`. It excludes credentials and third-party binary/model payloads. Use the recorded runtime lock when reproducing packaging.

Relevant source contracts:

- `app/desktop_bridge/router.py`
- `app/desktop_bridge/station.py`
- `app/requests`
- `app/auth/native_identity.py`
- `app/notifications`

The desktop package defines `npm run build`, `npm test`, `npm run test:browser` and its Tauri command. Python products retain their own `pyproject.toml`, tests and migrations; use the matching checkout's declared dependencies and test configuration. Do not run a sample server against a production profile. This documentation build validates documentation only and reuses the separately identified installed acceptance; it does not claim a new complete application CI run.

Recovery and support

Save work and use Core's canonical **Backup Now** before maintenance. That operation stops writers. Preserve the coherent profile/database/document backup and its receipt, then use **Start** to resume. A source export is not a full backup.

Use **Restore Latest Backup** through the supported controls. If the result says **Restore needs service start**, use **Start**, then **Check**, and reopen a known record. Do not repeat restore merely because services need to start. The fresh-profile acceptance verified that a fulfilled request survived restore and a post-backup marker did not.

Keep Station's dedicated database and media evidence together in its own backup. Core's native-profile backup must not be assumed to include an external Station data directory. Stop writers during a coordinated backup. Preserve failed media attempts and logs.

On failure, record the time, selected composition, component/installer versions and exact message. Check System Health for stopped, starting, ready or failed dependencies. Use the existing Retry control after readiness; do not create duplicate business records to refresh a stale view. Share only a redacted diagnostic bundle. Preserve secrets, original evidence, recovery material and unrelated installations.

Prepared package installation

The renamed distributions are not yet published on a package index. Use the matching verified candidate wheel directory, not a renamed historical release URL. Installation of these Python packages is a developer/package check; the signed Windows MSI and its installed acceptance remain separate.

In a new Windows virtual environment, replace the example wheel-directory path with the directory accompanying this candidate and verify its SHA256 manifest:

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
$candidateWheels = (Resolve-Path 'C:\path\to\candidate-wheels').Path
python -m pip install (Join-Path $candidateWheels 'townlight_core-1.3.0.dev2-py3-none-any.whl')
```

```
python -m pip install (Join-Path $candidateWheels
'townlight_records-1.8.0.dev2-py3-none-any.whl')
python -c "import townlight_records; from importlib.metadata import version;
print(version('townlight-records'))"
```

Expected principal import: `townlight_records`; expected distribution version: `1.8.0.dev2`. Use a fresh empty database/profile for workflow testing. Existing city data and the previous release remain untouched; no in-place upgrade or old-import compatibility shim is supplied.

License and responsible use

Current owned code is AGPL-3.0-only. Preserve full license, matching corresponding source, dependency notices and historical grants. Terms of use do not remove license rights. Source imports do not confer permission to publish third-party material. Review all automatic drafts and use the separate owning-product decision before official action. CivicCast remains independent.

Earlier documentation

Previous editions are retained under `docs/history/pre-townlight-beta-1/`. They describe their original software and license grants. Use the current Townlight Beta 1 manual for this candidate.